

Partie 4 : Forêt Aléatoire

Compléments de Mathématiques 2 – Introduction au Machine Learning

HAMLIL Mohamed

Invalid Date

Table des matières

1	Chargement des données	1
2	Forêt Aléatoire (Random Forest)	2
2.1	Principe	2
2.2	Justification	2
2.3	Traitement des données manquantes	2
2.4	Hyperparamètres	2
2.5	Résultats de la validation croisée	3
2.6	Importance des variables	4
2.7	Évaluation sur le jeu de test	5
3	Sauvegarde	7

1 Chargement des données

```
train_data <- readRDS(here::here("output", "train_data.rds"))
test_data <- readRDS(here::here("output", "test_data.rds"))
cat("Train :", nrow(train_data), "| Test :", nrow(test_data), "\n")
```

Train : 21033 | Test : 9013

2 Forêt Aléatoire (Random Forest)

2.1 Principe

Définition

La forêt aléatoire est un ensemble de B arbres de décision, chacun entraîné sur un échantillon bootstrap des données. À chaque nœud, seul un sous-ensemble aléatoire de m variables est considéré. La prédiction finale est obtenue par **vote majoritaire** (Breiman 2001).

Cette méthode réduit la variance par rapport à un arbre unique et capture naturellement les relations non-linéaires et les interactions entre variables.

2.2 Justification

- **Non-linéarité** : peut capturer des relations complexes entre les notes olfactives et la satisfaction.
- **Robustesse** : gère implicitement les données hétérogènes.
- **Importance des variables** : fournit un classement des variables les plus prédictives.
- **Peu de prétraitement** : pas besoin de centrer/réduire.

2.3 Traitement des données manquantes

Les valeurs manquantes ont été imputées par la médiane lors de la préparation. La forêt aléatoire est intrinsèquement robuste à ce type d'imputation.

2.4 Hyperparamètres

Nous optimisons :

- **mtry** : nombre de variables candidates à chaque nœud ($\sqrt{p}$ par défaut)

Nous utilisons la même **validation croisée 5-fold** avec l'**AUC**.

```
ctrl <- trainControl(  
  method = "cv",  
  number = 5,  
  classProbs = TRUE,  
  summaryFunction = twoClassSummary,  
  savePredictions = "final"
```

```
)

grid_rf <- expand.grid(
  mtry = c(3, 5, 7, 10, 15, 20)
)

set.seed(42)
model_rf <- train(
  Satisfaction ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl,
  tuneGrid = grid_rf,
  metric = "ROC",
  ntree = 500,
  importance = TRUE
)
```

2.5 Résultats de la validation croisée

```
cat("Meilleur mtry :", model_rf$bestTune$mtry, "\n")
```

Meilleur mtry : 7

```
cat("AUC (CV) :", round(max(model_rf$results$ROC), 4), "\n")
```

AUC (CV) : 0.7483

```
plot(model_rf, main = "Random Forest - Tuning de mtry")
```

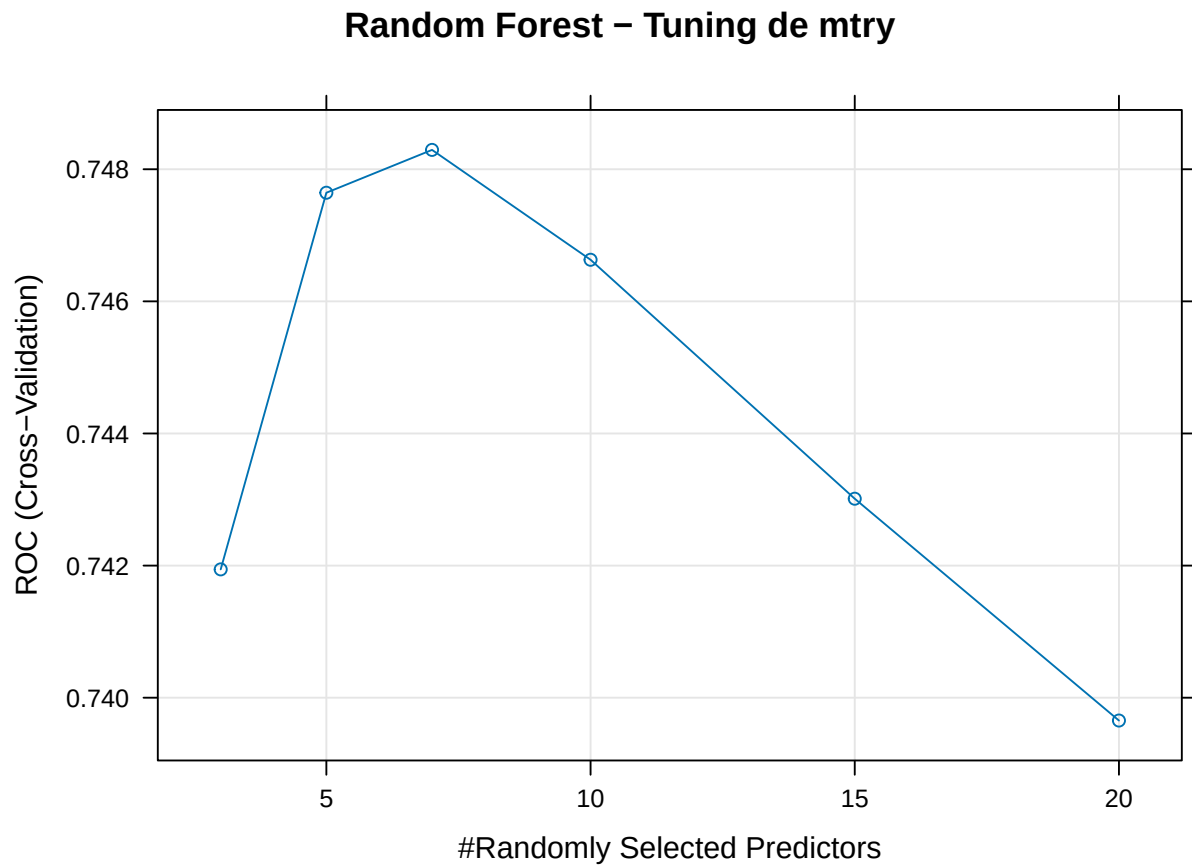


FIGURE 1 : Performance de la forêt aléatoire selon mtry

2.6 Importance des variables

```
var_imp <- varImp(model_rf)
plot(var_imp, top = 20, main = "Top 20 des variables les plus importantes")
```

Top 20 des variables les plus importantes

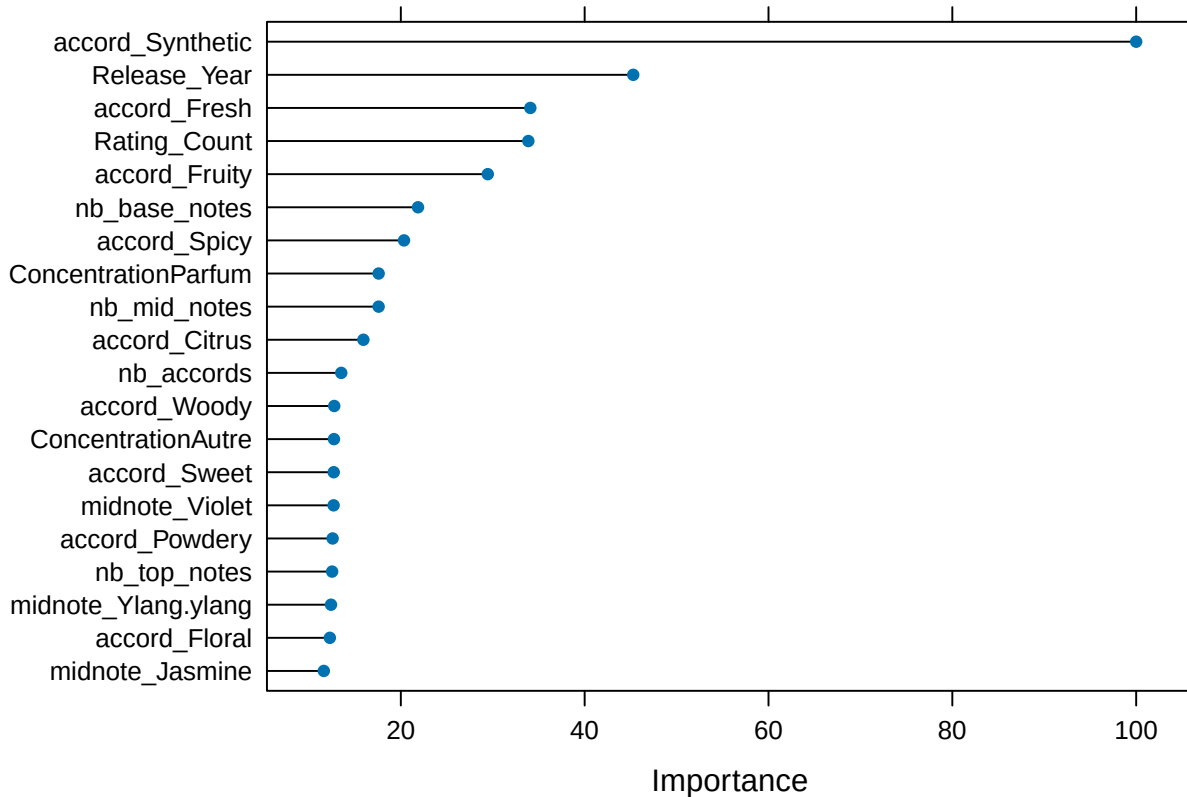


FIGURE 2 : Importance des variables (Random Forest)

💡 Interprétation

Le classement de l'importance des variables permet d'identifier quelles caractéristiques des parfums (accords, notes, concentration) influencent le plus la satisfaction des utilisateurs.

2.7 Évaluation sur le jeu de test

```
pred_rf <- predict(model_rf, test_data)
prob_rf <- predict(model_rf, test_data, type = "prob")

cm_rf <- confusionMatrix(pred_rf, test_data$Satisfaction, positive = "Oui")
print(cm_rf)
```

Confusion Matrix and Statistics

	Reference	
Prediction	Non	Oui
Non	2523	1102
Oui	1591	3797

Accuracy : 0.7012
 95% CI : (0.6916, 0.7106)
 No Information Rate : 0.5435
 P-Value [Acc > NIR] : < 2.2e-16

 Kappa : 0.3921

 McNemar's Test P-Value : < 2.2e-16

 Sensitivity : 0.7751
 Specificity : 0.6133
 Pos Pred Value : 0.7047
 Neg Pred Value : 0.6960
 Prevalence : 0.5435
 Detection Rate : 0.4213
 Detection Prevalence : 0.5978
 Balanced Accuracy : 0.6942

 'Positive' Class : Oui

```

roc_rf <- roc(test_data$Satisfaction, prob_rf$Oui)

plot(roc_rf, col = "darkgreen", main = "Courbe ROC - Forêt Aléatoire",
      print.auc = TRUE, print.auc.x = 0.4, print.auc.y = 0.3)
  
```

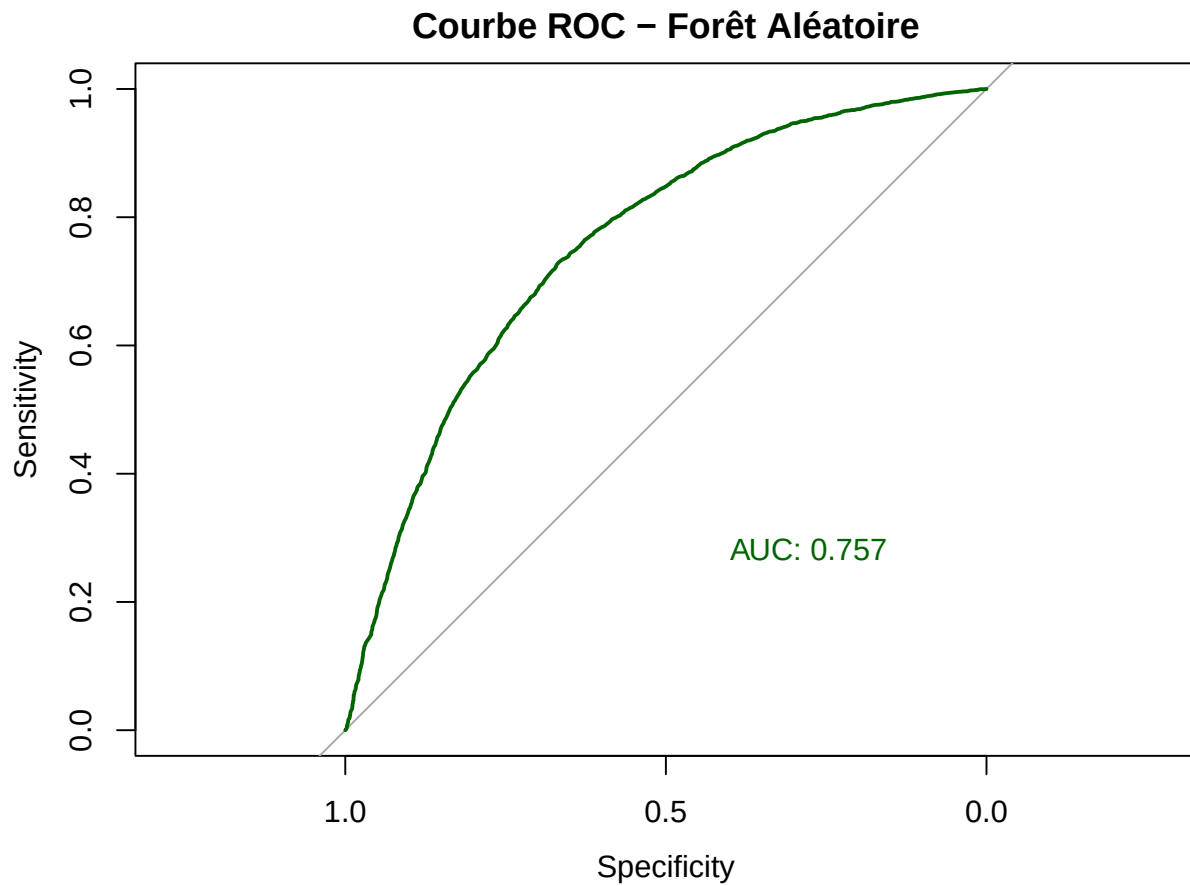


FIGURE 3 : Courbe ROC – Forêt Aléatoire

3 Sauvegarde

```
saveRDS(model_rf, here::here("output", "model_rf.rds"))
saveRDS(cm_rf, here::here("output", "cm_rf.rds"))
saveRDS(roc_rf, here::here("output", "roc_rf.rds"))
cat("Modèle et résultats sauvegardés dans output/\n")
```

Modèle et résultats sauvegardés dans output/

Breiman, Leo. 2001. « Random Forests ». *Machine Learning* 45 (1) : 5-32.